EXPT NO: 1 A)	LINEAR PROGRAMMING PROBLEM-CONSTRAINT OPTIMIZATION
DATE:	

AIM:

To solve a Linear Programming Problem (LPP) for constraint optimization using Python.

PROCEDURE:

1. **Install Required Libraries:**
o Make sure you have numpy and scipy installed. You can install them using pip if not already installed.
bash
Copy code
pip install numpy scipy

2. **Define the Problem:**
o Identify the objective function to maximize or minimize.
o Determine the constraints (equality and inequality).

3. **Set Up the Problem in Python:**
o Use the scipy.optimize.linprog function to set up and solve the LPP.

4. **Run the Code:**
o Execute the Python code to get the optimized result.

5. **Analyze the Output:**
o Interpret the results and validate the solution against the problem constraints

CODE

PROGRAM 1:

```
import numpy as np
from scipy.optimize import linprog

# Coefficients of the objective function (maximize 3x + 2y → minimize -3x - 2y)
c = [-3, -2]

# Inequality constraints (Ax ≤ b)
A = [
    [2, 1],
    [4, -5]
]
b = [20, 10]

# Bounds for x and y (x ≥ 0, y ≥ 0)
x_bounds = (0, None)
y_bounds = (0, None)

# Solve the linear programming problem
result = linprog(c, A_ub=A, b_ub=b, bounds=[x_bounds, y_bounds], method='highs')

# Print results
print("Optimal value:", -result.fun)
print("Values of x:", result.x)
```

OUTPUT

```
Optimal value: 40.0
Values of x: [ 0. 20.]
```

Problem 2:

```
import numpy as np
from scipy.optimize import linprog

# Coefficients of the objective function (Minimize 2x + 3y)
c = [2, 3]
```

```
# Convert ≥ constraints into ≤ by multiplying by -1
A = [
    [-1, -1],
    [-2, -1]
]
b = [-4, -5]

# Bounds for x and y (x ≥ 0, y ≥ 0)
x_bounds = (0, None)
y_bounds = (0, None)

# Solve the linear programming problem
result = linprog(c, A_ub=A, b_ub=b, bounds=[x_bounds, y_bounds], method='highs')

# Print results
print("Minimum value:", result.fun)
print("Values of x:", result.x)
```

OUTPUT

```
Minimum value: 10.0
Values of x: [1. 3.]
```

RESULT:

The Linear Programming Problem Has Been Completed Sucessfully

EXPT NO: 1 B)	**GRAPHICAL METHOD**
DATE:	

AIM:

To solve a Linear Programming Problem (LPP) using the graphical method and find the optimal solution using Python.

PROCEDURE:

- Import required libraries: numpy, matplotlib, and scipy.optimize.
- Define the objective function coefficients. Since linprog performs minimization, convert the maximization problem into minimization by taking negative values.
- Define the inequality constraints in matrix form (A and b).
- Specify the bounds for decision variables ($x \geq 0$, $y \geq 0$).
- Use the linprog() function to solve the LPP.
- Extract the optimal values of decision variables (x and y).
- Convert the result back to maximization to obtain the optimal value of Z.
- Generate values for x to plot constraint equations.
- Plot all constraint lines on a graph.
- Shade the feasible region that satisfies all constraints.
- Mark the optimal solution point on the graph.
- Label axes, add legend, grid, and display the graph.

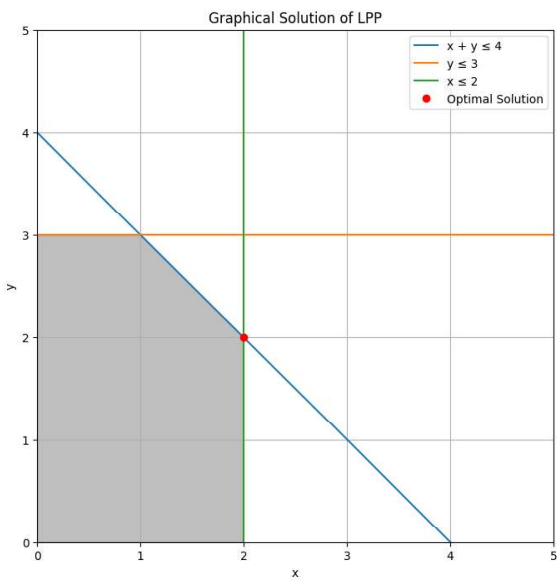
PROGRAM 1:

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.optimize import linprog
# Objective function (maximize Z = 3x + 2y → convert to minimize)
c = [-3, -2]
# Inequality constraints (Ax ≤ b)
A = [
    [1, 1],   # x + y ≤ 4
    [0, 1],   # y ≤ 3
    [1, 0]    # x ≤ 2
]
b = [4, 3, 2]
# Bounds for x and y
x_bounds = (0, None)
y_bounds = (0, None)
# Solve using linprog
res = linprog(c, A_ub=A, b_ub=b, bounds=[x_bounds, y_bounds], method='highs')
# Extract solution
x_opt, y_opt = res.x
z_opt = -res.fun  # Convert back to maximization
print(f"Optimal solution: x = {x_opt}, y = {y_opt}, Z = {z_opt}")

# Plotting
x = np.linspace(0, 5, 400)
y1 = 4 - x
y2 = np.ones_like(x) * 3
y3 = np.ones_like(x) * 2
plt.figure(figsize=(8, 8))
plt.plot(x, y1, label='x + y ≤ 4')
plt.plot(x, y2, label='y ≤ 3')
plt.plot(y3, x, label='x ≤ 2')
plt.fill_between(x, np.minimum(y1, y2), where=(x <= 2), color='grey', alpha=0.5)
plt.plot(x_opt, y_opt, 'ro', label='Optimal Solution')

plt.xlim(0, 5)
plt.ylim(0, 5)
plt.xlabel('x')
plt.ylabel('y')
plt.title('Graphical Solution of LPP')
```

```
plt.legend()
plt.grid(True)

plt.show()
```

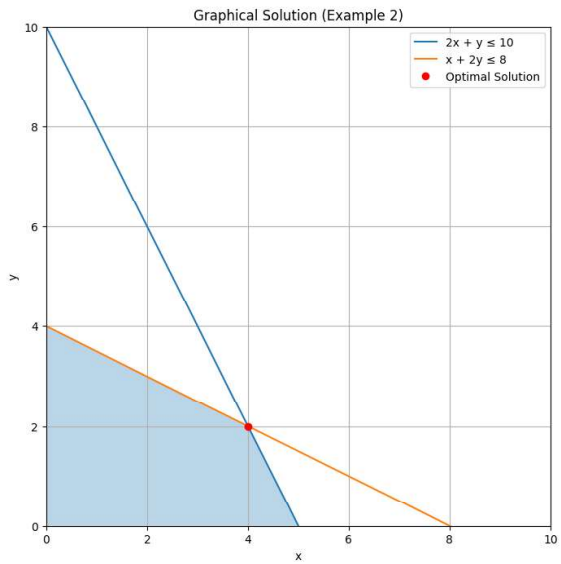
OUTPUT:

Optimal solution: x = 2.0, y = 2.0, Z = 10.0



PROBLEM 2:

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.optimize import linprog
# Objective function (maximize Z = 5x + 4y)
c = [-5, -4]
# Constraints
A = [
    [2, 1],   # 2x + y ≤ 10
    [1, 2]    # x + 2y ≤ 8
]
b = [10, 8]
# Bounds
x_bounds = (0, None)
y_bounds = (0, None)
# Solve
res = linprog(c, A_ub=A, b_ub=b, bounds=[x_bounds, y_bounds],
method='highs')
x_opt, y_opt = res.x
z_opt = -res.fun
print(f"Optimal solution: x = {x_opt}, y = {y_opt}, Z = {z_opt}")
# Plot
x = np.linspace(0, 10, 400)
y1 = (10 - 2*x)
y2 = (8 - x) / 2
plt.figure(figsize=(8, 8))
plt.plot(x, y1, label='2x + y ≤ 10')
plt.plot(x, y2, label='x + 2y ≤ 8')
plt.fill_between(x, np.minimum(y1, y2), where=(y1 >= 0) & (y2 >= 0),
alpha=0.3)
plt.plot(x_opt, y_opt, 'ro', label='Optimal Solution')
plt.xlim(0, 10)
plt.ylim(0, 10)
plt.xlabel('x')
plt.ylabel('y')
plt.title('Graphical Solution (Example 2)')
plt.legend()
plt.grid(True)

plt.show()
```

OUTPUT:

Optimal solution: x = 4.0, y = 2.0, Z = 28.0